# Routed Recall Does Not Degrade at a Trillion Rows [Experiment, Analysis & Benchmark]

Routed search over compressed vectors, measured against the exact scan, on a shared research cluster

Andrew H. Bond
San José State University
San José, California, USA
andrew.bond@sjsu.edu

## ABSTRACT

We measure routed inverted-file search over one trillion 4-bit quantized vectors against the exact scan of the same compressed index. The index is 500 servers of two billion rows, 24 TB, built from a seeded synthetic corpus on the National Research Platform's shared Kubernetes cluster. Probing 32 of 2048 cells, routed search returns 0.989 of the exact top ten for queries that are corpus rows and 0.969 for queries the index does not contain, and probing 128 cells it returns 0.999 for both. PENDING-1TRG-ABSTRACT. Within the trillion-row index, subsets of servers that differ only in their number of rows show no loss of recall as rows are added, for both query sets, and a gain at narrow widths, the behaviour the Faiss authors describe for a fixed coarse quantizer, here measured at a trillion rows. A prediction from twenty servers, registered before the non-member run was scored, matched its trillion-row recall within error, and a second, from separate corpora of $10^8$ to $10^9$ rows, failed at 32 probes. Because a probed cell is scored as the scan scores it, recall at any width is the share of reference neighbours in the probed cells, and that share, computed from cell metadata, matched every measured width, three of them measured after it was recorded. The same identity compares cell orders at equal work: plain centroid order does as well as the router's radius-adjusted order, so the result is a property of the ordinary inverted file. Holding recall at 32 probes costs a median of 19 billion rows scored per query, 1.9 percent of the index, on all 500 servers. Against float rows regenerated from the seed the compressed ranking keeps at most 0.621 of the true neighbours, so the result is about routing and not about compression. We describe the jobs, the pool and the accounting of the run, and every number can be recomputed from the archived record.

**PVLDB Reference Format:**
Andrew H. Bond. Routed Recall Does Not Degrade at a Trillion Rows [Experiment, Analysis & Benchmark]. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://doi.org/10.5281/zenodo.23608470.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

## 1 INTRODUCTION

Approximate nearest-neighbour search over compressed vectors is routine at a billion rows [5, 12, 14, 20], and indexes of a trillion vectors have been built. The Faiss library paper [6] describes an index of 1.5 trillion vectors built in 2000 shards on a few hundred servers and searched in about a second per query, and UBASE [21] serves a production deployment of about a trillion vectors. Neither reports recall at that scale. UBASE reports recall on sets of up to about a hundred million vectors and latency and memory at a trillion, and the Faiss account reports the build and the search time. What is missing at that scale is the comparison that defines the error of routing, the routed result against an exact scan of the whole index.

What to expect has a precedent. The Faiss authors observe that with a fixed coarse quantizer a growing dataset can only bring a query's nearest neighbour closer, that a closer neighbour is more likely to sit in a cell near the query, and that the number of probes needed for a target accuracy then falls with the dataset size [6, Section 5.1]. They report it in the regime where the number of cells is larger than optimal for the dataset, with $2^{18}$ cells for five million vectors as their example, and measure it as search time at fixed accuracy. This paper does not offer that mechanism as new. It measures the same direction far on the other side of that regime, 2048 cells for $10^{12}$ rows, nearly half a billion rows a cell, against an exact scan, inside one index where nothing but the number of rows changes, and leaves a record from which every number can be recomputed. Those three things, the trillion-row exact comparison, the controlled scale series and the record, are the contribution.

This paper makes that comparison at $10^{12}$ rows. When an inverted-file index is partitioned into shards that share one coarse quantizer and one rotation, and the corpus grows by adding shards, does the routing lose anything it would have found by scanning everything? We built the whole index, scanned all of it exactly, ran the routed search over the same shards at five probe widths, and compared. Three things make the comparison more than one number. The exact scan of any subset of servers is the merge of that subset's partial results, so one index gives a scale series in which only the number of rows changes. Recall at any width follows from where the reference neighbours sit in the router's probe order, so the curve over widths can be computed from metadata and checked against routed passes. And the corpus is a seed, so every row can be regenerated and every number recomputed from the record.

The findings are these. Routed recall against the exact scan is 0.989 at 32 probes and 0.999 at 128 probes at $10^{12}$ rows for queries that are corpus rows, with 95 percent bootstrap intervals over the

100 queries of 0.982 to 0.995 and 0.997 to 1.000, and 0.969 and 0.999 for 100 queries the index does not contain (Section 4). PENDING-1TRG-INTRO. Inside the trillion-row index recall moves by less than 0.01 at 128 probes as servers are added and rises at 16 and 32 probes, and for the non-member queries, where no subset changes whether a query is its own neighbour, it rises at both widths measured (Section 4.2). The recall curve over nine widths computed from the cell ranks of the reference neighbours matched the five widths measured, three of them measured after the curve was recorded (Section 4.4). At equal rows scored, plain centroid order gives the same recall as the router's radius-adjusted order or better, so nothing here depends on that adjustment (Section 4.5). Holding that recall costs a fixed share of the index per query, a median of 19 billion rows at 32 probes (Section 4.8). For an index designer the lesson is in two parts. A probe width tuned on a small slice of a corpus that grows under a fixed coarse quantizer did not have to be widened as the corpus grew five-hundredfold inside one index, and the work that width buys grows with the corpus, so keeping work per query bounded would take more cells, not a narrower width, which this paper does not test. Whether the trillion-row figure could be known in advance we tested by registration: a deficit law fitted on twenty servers predicted the non-member recall at $10^{12}$ rows within its error at both widths, and a model chosen on separate corpora of $10^8$ to $10^9$ rows failed at 32 probes (Section 4.3). The limit is that the corpus is synthetic, of intrinsic dimension about 16, and the reference is the compressed index's own exact scan. Section 7 gives the figures that bound the result, an upper bound of 0.621 on the compressed ranking's true recall at $10^{12}$ rows and a real-embedding pilot.

The index was built and measured on the National Research Platform's Nautilus cluster [23], which many research groups share and on which nothing is reserved. Every build job and almost every measurement job ran in the cluster's smallest job class, one CPU and two gigabytes. Section 5 describes how, and Section 6 accounts for what the run used.

## 2 THE INDEX AND THE CORPUS

*Index.* Each row is a 32-dimensional vector projected to 24 dimensions by one basis shared by the whole fleet, quantized to 4 bits per dimension, and assigned to one of 2048 inverted-file cells by one shared coarse quantizer. Both the basis and the quantizer are fitted once, on a bootstrap shard, and read by every build job. A shard holds five million rows. A server holds 400 shards, two billion rows, on one block volume. The on-disk row is 12 bytes of codes, 8 bytes of per-row norms used by the asymmetric distance computation, and 4 bytes of cell membership. Every server reports 48 009 656 624 index bytes for its two billion rows, 24.0 bytes per row, so the fleet holds 24.0 TB of index, on volumes provisioned at 56 GiB each, 30.1 TB in all. Scoring is asymmetric distance computation over the codes, in row blocks, from memory-mapped shards. Because the basis and the coarse quantizer are shared, scores from different shards are comparable and a global top-$k$ is the exact merge of per-shard top-$k$s (Figure 1). That property is what makes the measurement below exact rather than approximate.

*Routing.* A query is rotated by the shared basis and compared with the 2048 cell centroids. A cell's angular radius $r$ is the largest

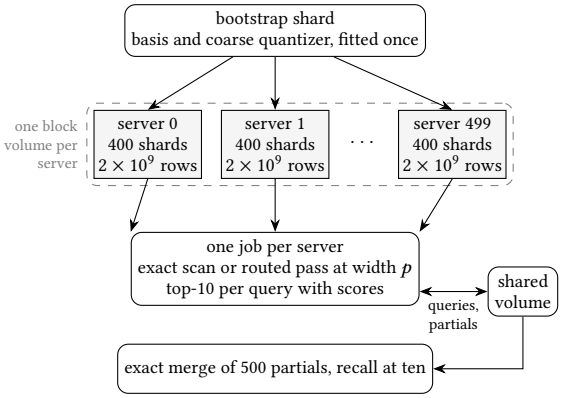



**Figure 1: The fleet. One bootstrap shard fixes the projection basis and the 2048-cell coarse quantizer for every shard on every server, so scores are comparable across servers and the global top-10 is the exact merge of per-server top-10s. Each phase is one job per server, and each job reads only its own volume and the shared volume.**

angle between its centroid and the quantized direction of a bootstrap row assigned to it. Each cell gets the priority $\cos(\max(0, \theta - r/2))$, $\theta$ the angle from the query to the centroid, and the routed search scores every row of the $p$ cells of highest priority, on every server, with the same asymmetric distance the exact scan uses. The radii are the bootstrap's, so every server probes the same cells for a query. The priority is an ordering heuristic and not a bound. With the full radius it would bound the cosine of the bootstrap's own rows by the triangle inequality on the sphere, but half the radius does not, the radius fitted on one shard need not cover the rows of later shards, and the asymmetric score is not that cosine. Nothing below depends on it being a bound. Section 4.5 compares it with plain centroid order and the full radius at equal work.

*Corpus.* The corpus is defined by a seed and regenerated on demand. Each global shard draws a fresh rank-16 Gaussian basis, projects Gaussian coefficients through it with a linear decay of singular values from 1.0 to 0.3, and adds five percent noise, in an ambient dimension of 32. A shard is a pure function of its global index, so any job can produce any shard without reading anything, and the build moved no corpus bytes. A build from files would first have placed 128 TB of float vectors where 500 jobs could read them. A single shard is rank 16 plus noise by construction. Across shards the intrinsic dimension measures 15.4 to 17.0 by the two-nearest- neighbour estimator [7] on unit-normalized samples of 25000 to 200000 rows drawn across shards, with no trend whether the rows grow within a fixed set of shards or as 5 to 40 new shards. These are mixed-shard estimates, and no within-shard estimate was made. Real embeddings such as Cohere Embed v3 at 1024 dimensions measure 52 to 61 by the same estimator. Hubness, the concentration of nearest-neighbour lists on a few points [17], rises with corpus size on this recipe, at about 0.42 per decade at $k = 10$ against roughly 0.25 per decade on real data. We attribute the rise to how shards are added. Each shard draws its own random 16-dimensional subspace, two random 16-dimensional subspaces of a

**Table 1: The phases of a measurement. Every job writes its output under a temporary name and renames it, and exits at once if the output exists. The metadata phases read only partials and the index's cell metadata.**

| phase | jobs | reads | writes |
|---|---|---|---|
| query cache | 1 | seed | 100 queries |
| exact scan | 500 | own volume, queries | top-10 per query |
| routed pass | 500 | own volume, queries | top-10 per width |
| score | 1 | 1500 partials | recall at ten |
| second sweep | 500 | own volume, queries | top-10 at 16, 64, 256 |
| cell census | 500 | own cell metadata | cell counts |
| census merge | 1 | counts, partials | cell ranks |

32-dimensional space are not orthogonal, and so the rows of a new shard land close to rows of shards already present. Hubness can act on the result. Recall depends on whether the router reaches the cells of the reference neighbours (Section 4.4), and hubness changes which rows become those neighbours and so which cells they sit in. The identity of Section 4.4 does not separate that effect, and we did not measure it. A corpus with the hubness of real embeddings could place its neighbours differently.

*Scale points.* The same protocol has been run at $10^8$ rows on 4 servers, $10^9$ on 4, $10^{10}$ on 8, $10^{11}$ on 50 and $10^{12}$ on 500. Per-server work is constant from $10^{11}$ on, two billion rows, so the step from 50 to 500 servers changes nothing per node and changes only the number of chances to draw a slow or failing one.

## 3 PROTOCOL

A measurement is a set of phases, each a pool of independent jobs (Table 1). The query-cache phase writes the query set to the shared volume once, and every server reads it from that file. The reference phase is one job per server that scans the server's two billion rows exactly and writes its top-10 per query with scores. The routed phase is one job per server that runs the routed search at 32 and 128 probes and writes the same shape of partial. The score phase merges the 500 reference partials into the exact global top-10, merges the routed partials the same way, and reports recall at ten of the routed set against the reference set, averaged over queries. A second routed sweep at 16, 64 and 256 probes was submitted the day the first score landed and scored two days later, and two metadata phases read only the partials and the cell metadata (Section 4.4). The score program has not changed since the $10^9$ run.

Three query sets were measured. The corpus-row set is the first 25 rows of each of four seeded shards, 0, 50000, 100000 and 150000, spread evenly across the index, and the same recipe gave the queries at every earlier scale point, so the comparison across sizes holds the query recipe fixed. The non-member set is the first 25 rows of each of shards 200000, 250000, 300000 and 350000 of the same generator, seeds no corpus shard uses, so no query is in the index. Both draw from four shards, and each shard has its own subspace, so both cover four subspaces of the generator. The many-shard set covers two hundred. It is row 0 of each of shards 400000 to 400099, none in the index, and row 0 of one shard on each of 100 servers spread across the index, all in it. It was measured after the index volumes were released, on an index regenerated from the seed (Section 4.7).

**Table 2: Recall at ten of routed search against the exact full scan of the same compressed index, by corpus size, corpus-row queries. Queries are 100 at $10^8$, $10^9$ and $10^{12}$ rows and 500 at $10^{10}$ and $10^{11}$. The last column is the median over servers of the time of one server's exact scan call. The $10^{12}$ jobs ran on one CPU each (164 exact scans at six, Section 5), where the earlier runs had six, so that column is not comparable across rows.**

| rows | servers | 32 probes | 128 probes | median scan (s) |
|---|---|---|---|---|
| $10^8$ | 4 | 0.979 | 0.999 | 47 |
| $10^9$ | 4 | see text | | 589 |
| $10^{10}$ | 8 | 0.982 | 0.999 | 8495 |
| $10^{11}$ | 50 | 0.983 | 0.999 | 19041 |
| $10^{12}$ | 500 | 0.989 | 0.999 | 2664 |

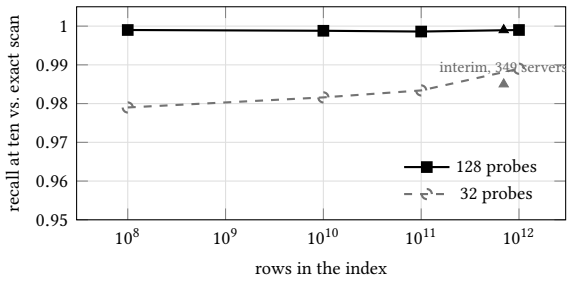



**Figure 2: Recall against the exact scan by corpus size, corpus-row queries. The triangles are the interim score taken while the $10^{12}$ run was two thirds through its routed phase, on the 349 servers that had finished both phases, 698 billion rows.**

## 4 RESULTS

### 4.1 Recall across four orders of magnitude

Table 2 and Figure 2 give the number the run exists to produce. At $10^{12}$ rows, routed search at 128 probes of 2048 cells returns 0.999 of the exact scan's top ten, and at 32 probes 0.989. At 128 probes the figure is 0.999, 0.9988 and 0.9986 at the three smaller scales, a spread of 0.0004. At 32 probes it rises slowly with scale, 0.979, 0.982, 0.983, 0.989, and Section 4.2 traces that rise to the number of rows. The interim score, on the first 349 servers to finish both phases, gave 0.985 and 0.999. At 32 probes 90 of the 100 queries have recall 1.0, nine have 0.9 and one has 0.8. At 128 probes 99 queries have recall 1.0 and one has 0.9, so the mean of 0.999 is one query missing one neighbour. Resampling the 100 queries gives 95 percent bootstrap intervals of 0.982 to 0.995 at 32 probes and 0.997 to 1.000 at 128. These intervals describe variation within this query sample. They say nothing about other shards' subspaces, which Section 4.7 measures. The earlier scale points recorded only the mean, so they carry no interval.

The $10^9$ row has no routed-versus-scan figure in the table because that run carried a cold store of the original float vectors and

**Table 3: Recall at ten against the exact scan of a subset of $k$ servers of the trillion-row index, the mean of five seeded random subsets of $k$ servers (at $k = 500$, the whole index), for the corpus-row queries at four widths and the non-member queries at the two widths their run measured. Only the number of rows changes along $k$. Eight of the twelve values of $k$ are shown, and Figure 3 plots all twelve.**

| | | corpus rows, probes | | | | non-members | |
|---|---|---|---|---|---|---|---|
| servers | rows | 16 | 32 | 64 | 128 | 32 | 128 |
| 1 | $2 \times 10^9$ | 0.797 | 0.905 | 0.965 | 0.991 | 0.843 | 0.980 |
| 5 | $10^{10}$ | 0.846 | 0.931 | 0.979 | 0.996 | 0.890 | 0.990 |
| 10 | $2 \times 10^{10}$ | 0.861 | 0.943 | 0.984 | 0.997 | 0.908 | 0.993 |
| 50 | $10^{11}$ | 0.906 | 0.970 | 0.990 | 0.998 | 0.940 | 0.996 |
| 100 | $2 \times 10^{11}$ | 0.913 | 0.971 | 0.993 | 0.999 | 0.947 | 0.998 |
| 200 | $4 \times 10^{11}$ | 0.942 | 0.984 | 0.996 | 1.000 | 0.959 | 0.998 |
| 300 | $6 \times 10^{11}$ | 0.948 | 0.988 | 0.997 | 0.999 | 0.964 | 0.999 |
| 500 | $10^{12}$ | 0.952 | 0.989 | 0.997 | 0.999 | 0.969 | 0.999 |

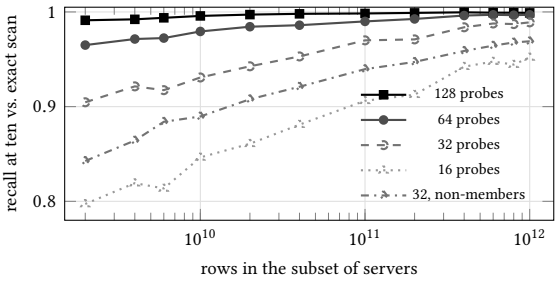



**Figure 3: Recall against the exact scan of $k$ servers of the one index, mean of five random subsets, from two billion to a trillion rows, corpus-row queries and, dash-dotted, the non-member queries at 32 probes. Wide probes are flat, and narrow probes gain as rows are added.**

measured true recall instead. The routed shortlist at 128 probes returned 0.999 of the compressed scan there too, the compressed scan itself found only 0.592 of the true neighbours, and reranking the shortlist against the stored originals raised that to 0.991. That row is the reason Section 7 exists.

## 4.2 Recall inside one index as the rows grow

The runs of Table 2 differ in more than size. Each was a separate index with its own bootstrap shard, coarse quantizer and queries, so a change between them could come from any of those. The trillion-row run removes that confound. The exact top ten over any subset of servers is the merge of that subset's partials, and so is the routed top ten, so recall against the exact scan of any subset is exact and needs no new scan. A subset of $k$ servers is a corpus of $2k$ billion rows from the same generator, with the same basis, coarse quantizer, router and queries, and along $k$ only the number of rows changes. We took the first $k$ servers and five seeded random subsets of $k$ servers for twelve values of $k$ from 1 to 500. The random subsets are drawn uniformly from all servers. Up to $k = 20$ none of them holds any of the four servers the queries came from, servers 0, 125, 250 and 375, and from $k = 50$ on they hold up to four. The first $k$ servers always include server 0.

Table 3 and Figure 3 give the result. At 128 probes recall moves from 0.991 at two billion rows to 0.999 at a trillion, and at 256 probes from 0.999 to 1.000. At narrow widths recall rises with the number of rows, at 16 probes from 0.797 to 0.952 and at 32 probes from 0.905 to 0.989. Between one and ten servers, where no random subset holds a server the queries came from, recall at 32 probes already rises from 0.905 to 0.943, so the rise does not depend on those servers. The rise is larger than the spread between subsets of one size. At 32 probes the five subsets of one server range from 0.898 to 0.908, of ten servers from 0.935 to 0.951, and of fifty from 0.960 to 0.980.

The corpus-row queries carry one more confound, since a subset that holds a query's own row returns it as that query's nearest neighbour. The non-member queries are rows of no shard, so for them no subset changes that, and their partials give the same series with no new scan (Table 3, last two columns). At 32 probes their recall rises from 0.843 at one server to 0.969 at five hundred, with the five subsets of one server between 0.832 and 0.851, of ten between 0.897 and 0.927 and of a hundred between 0.939 and 0.957. At 128 probes it rises from 0.980 to 0.999. Growing the corpus from two billion to a trillion rows therefore causes no measurable loss of routed recall at any width under this protocol, for either query set, and at narrow widths it raises it. The rise at 32 probes across the separate runs, 0.979, 0.982, 0.983 and 0.989, is the same effect with the confound present.

**Table 4: Registered predictions of routed recall at $10^{12}$ rows for the non-member queries, committed before that run was scored. Error is prediction minus measured value, with the standard error of the difference from a paired bootstrap over the 100 queries (2000 resamples). A prediction passes when its error is within two standard errors.**

| calibration rows | probes | model | pred. | error (SE) | |
|---|---|---|---|---|---|
| in index, $2 \times 10^9$–$4 \times 10^{10}$ | 32 | M1 | 0.953 | −0.016 (0.010) | pass |
| | 128 | M0 | 0.994 | −0.005 (0.003) | pass |
| separate, $10^8$–$10^9$ | 32 | M2 | 1.000 | +0.031 (0.007) | fail |
| | 128 | M1 | 0.998 | −0.001 (0.001) | pass |

## 4.3 Predicting the trillion-row value from a slice

The nested series shows what happened. Whether a designer could have known it in advance is a separate question, and we tested it before the non-member run was scored. Two registrations, each committed to the repository with its models, its selection rule, its pass rule and its grader script, predicted that run's routed recall at $10^{12}$ rows (Section 9). The models were M0, recall at the largest calibration size carried unchanged; M1, a power law in the deficit, $\log(1 - R)$ linear in $\log N$; and M2, recall linear in $\log N$, capped at 1. Each registration chose its model per width on the corpus-row queries, already scored, by the smallest error at $10^{12}$. The first calibrated on nested subsets of one to twenty servers of this index, from $2 \times 10^9$ to $4 \times 10^{10}$ rows, computed from those twenty servers' partials alone and committed before the score. The second calibrated

**Table 5: Recall at ten predicted from the cell ranks of the reference neighbours, and recall measured by a routed pass at that width, $10^{12}$ rows, corpus-row queries. The 32 and 128 columns were measured first. The 16, 64 and 256 columns were predicted and recorded before the sweep that measured them was submitted. The last row counts the queries whose predicted recall is 1.0.**

| probes | 1 | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 |
|---|---|---|---|---|---|---|---|---|---|
| predicted | 0.406 | 0.599 | 0.727 | 0.867 | 0.952 | 0.989 | 0.997 | 0.999 | 1.000 |
| measured | | | | | 0.952 | 0.989 | 0.997 | 0.999 | 1.000 |
| queries with recall 1.0 | 1 | 4 | 15 | 37 | 68 | 90 | 97 | 99 | 100 |

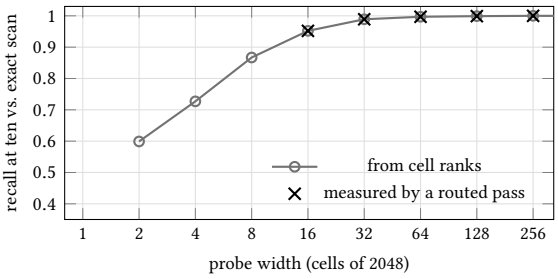



**Figure 4: The recall curve over probe width computed from the cell rank of each reference neighbour (line, from the census merge, no distance scan) and the five widths measured by routed passes over the whole index (crosses).**

on separately built corpora of $10^8$ to $10^9$ rows, shards of this index rebuilt from their seeds with its basis, coarse quantizer and router.

Table 4 gives the grades against the measured 0.969 and 0.999. The first registration passed at both widths. At 32 probes its fit on at most four percent of the index predicted 0.953, and carrying the twenty-server value unchanged would have predicted 0.914. The second failed its primary test. Its development comparison chose M2 at 32 probes because the cap at 1 sat close to the corpus-row value of 0.989, a risk its registration stated, and on the non-member queries a prediction of 1 is 4.4 standard errors too high. Its M1 fit on the same small corpora predicted 0.953, but M1 was not its registered model at that width, so we report that only as context. At 128 probes both passed. For a designer this is one positive result and one negative one: a deficit law fitted on subsets of the index itself carried to a corpus twenty-five times larger, and a model chosen on separate small corpora did not, as registered. One index, one generator and 100 queries bound both.

## 4.4 Recall from the probe order alone

Every reference neighbour lives in one of the 2048 cells, and each query orders the cells by the priority the router uses. Inside a probed cell the routed pass scores every row with the same asymmetric distance the exact scan used, so a row gets the same score in both passes up to the order of floating-point additions, and a reference neighbour is returned exactly when its cell is probed, as long as that difference cannot move a row across the tenth score. The audit of the archived partials (Section 5) measured both. Of the 2.3 million rows that appear in a server's reference partial and one of its routed partials, 74 percent have bit-equal scores and the rest differ by at most 7 units in the last place, $4.2 \times 10^{-7}$, while the smallest gap between the tenth and eleventh merged reference scores of any query is $1.6 \times 10^{-5}$ for the corpus-row queries and $2.1 \times 10^{-6}$ for the non-members, with no ties. Recall at width $p$ is the share of reference neighbours whose cell has rank below $p$ in its query's order. That identity, stated over finite sets with exhaustive in-cell scoring as its hypothesis, is checked in Lean 4 with Mathlib in the repository (`lean/Pvldb1t/Reachability.lean`, theorem `recall_eq_share`). The proof establishes the set relationship under its hypotheses and nothing more. It takes equal scores as a hypothesis, and the measured margin above is what carries it to these data. It needs the reference neighbours, so it does not predict which rows become neighbours as the corpus grows, and it says nothing about other corpora or query populations.

What it gives is an instrument and a check. The share needs the cell of each of the 1000 reference neighbours and nothing else, so it was computed from the index metadata in 500 jobs of half a minute each and one merge, with no distance scan. Table 5 and Figure 4 give it at nine widths. It agreed with the two widths already measured, 32 and 128, to the last digit. We then recorded the values at 16, 64 and 256, submitted a routed pass at those widths on all 500 servers, and the measured recall came back 0.952, 0.997 and 1.000, equal to the recorded values at every width. That agreement validates the implementation of the routed pass, the merge and the census against one another. The neighbours' cell ranks have median 1, ninetieth percentile 9, ninety-ninth percentile 32 and maximum 169, so the one neighbour that 128 probes miss sits in the 170th cell of its query's order.

The identity also frames the stability across scale. The corpus grows by adding shards under one shared coarse quantizer, so an added shard puts rows into the same 2048 cells and leaves every query's probe order unchanged. Recall at width $p$ changes only if the ten nearest rows move to cells of different rank. Nothing forces nearer rows into earlier cells, since a row near a cell boundary can be close to the query and sit in a cell the priority ranks late. Whether they do is an empirical question. By the identity, recall at width $p$ is the share of reference neighbours in the first $p$ cells, so the nested series measures that share as rows are added, and on this index it rises at narrow widths and does not fall at any width. That is an observation about this corpus and router, not a consequence of the identity, and a corpus whose new rows land near cell boundaries could behave otherwise.

## 4.5 Does the cell order matter?

The router adjusts each centroid angle by half the cell's radius, and a reader can ask whether the result depends on that choice. The identity answers it without a scan. The census holds the cell of every reference neighbour and the size of every cell, so for any cell order and any budget of rows, recall is the share of neighbours whose cells fit in the budget. We gave each query, at each width, the rows the fleet's order scored there and let three orders spend them: plain centroid order, the fleet's order with half the radius, and the full radius. The fleet's order at its own budget reproduces the measured recall at every width, which checks the computation.

**Table 6: Recall at ten of three cell orders at equal work, $10^{12}$ rows, corpus-row queries. For each query and width $p$, every order probes cells in its own order while the rows scored stay within what the fleet's order scored at width $p$ (median given in billions). The fleet column reproduces the measured recall. The last column is the paired difference, fleet order minus centroid order, with a 95 percent bootstrap interval over queries. Computed from the cell census and the reference neighbours' cells by the identity of Section 4.4, with no distance scan.**

| probes | rows | centroid | fleet, $r/2$ | full $r$ | fleet − centroid |
|---|---|---|---|---|---|
| 8 | 5.0 | 0.901 | 0.867 | 0.569 | −0.034 (−0.055, −0.016) |
| 16 | 10.2 | 0.971 | 0.952 | 0.736 | −0.019 (−0.031, −0.009) |
| 32 | 19.2 | 0.992 | 0.989 | 0.860 | −0.003 (−0.008, 0.001) |
| 64 | 37.4 | 0.996 | 0.997 | 0.980 | 0.001 (0.000, 0.003) |
| 128 | 72.0 | 1.000 | 0.999 | 0.998 | −0.001 (−0.003, 0.000) |

Table 6 gives the result. At equal work plain centroid order is as good as the fleet's order or better, by 0.034 at 8 probes and 0.019 at 16, with intervals that exclude zero, and the two agree within 0.003 from 32 probes on. The full radius is worse at every width, 0.736 against 0.971 at 16 probes, and we did not examine why. The radius adjustment therefore contributes nothing to the recall reported here, and the results of this paper describe the ordinary inverted file that probes cells by centroid distance. This control covers the corpus-row queries only, because the census recorded the cells of their neighbours and not of the non-members'.

## 4.6 The cells, query placement, and queries outside the index

The same 500 metadata jobs took a census of the cells. The 2048 cells hold between 49.2 million and 2.15 billion rows, with mean 488 million and median 418 million, a Gini coefficient of 0.328, the largest one percent of cells holding 3.7 percent of rows, and no empty cell.

The 1000 reference neighbours of the corpus-row queries sit on 233 of the 500 servers, and one server holds 16.6 percent of them, because each shard's own basis keeps a query's neighbours near its home shard. Losing one server at random would cost 0.002 of recall in expectation, and losing that one server would cost 0.166. Because every query is a corpus row, it is also its own nearest neighbour.

The non-member set removes both. Its reference scan, routed passes at 32 and 128 probes and exact merge were rerun over all 500 servers with the programs of the first run. Its 1000 reference neighbours sit on 404 of the 500 servers, no server holds more than 1.0 percent of them, and the median query has its ten neighbours on eight servers. Routed recall against the exact scan was 0.969 at 32 probes and 0.999 at 128, with 95 percent bootstrap intervals of 0.954 to 0.982 and 0.997 to 1.000. At 32 probes 78 of the 100 queries had recall 1.0 and the lowest had 0.6. At 128 probes the two sets agree. At 32 probes the queries the index does not contain lose 0.020, which we attribute to the two advantages a corpus row has, its own row among its ten neighbours and neighbours in the cells its own shard's basis favours. Section 4.7 separates the two.

**Table 7: Rows scored per query by the routed search at $10^{12}$ rows, in billions, median with tenth and ninetieth percentiles over the 100 queries of each set, beside the figure balanced cells would give, $p/2048$ of the index. Every query touches all 500 servers. Bytes touched are 24 per row scored, and the last column is the median for corpus-row queries. Computed from the cell census and each query's probe order.**

| probes | balanced | corpus rows | non-members | TB |
|---|---|---|---|---|
| 16 | 7.8 | 10.2 (7.0–13.7) | 9.5 (7.2–13.6) | 0.24 |
| 32 | 15.6 | 19.2 (13.6–25.8) | 18.8 (14.0–24.9) | 0.46 |
| 64 | 31.3 | 37.4 (27.4–46.5) | 36.6 (28.0–46.0) | 0.90 |
| 128 | 62.5 | 72.0 (58.2–86.0) | 70.1 (56.2–85.8) | 1.73 |
| 256 | 125.0 | 138 (119–158) | 136 (117–155) | 3.31 |

## 4.7 Queries from two hundred shards

Both four-shard sets sample four of the generator's subspaces, and a bootstrap over their queries cannot show how recall varies across subspaces. The many-shard set samples two hundred, one query per shard, half of them corpus rows from shards on 100 different servers and half from seeds no corpus shard uses. By the time it was drawn the 500 index volumes had been released. The corpus is a seed, so we measured it on a regenerated index. Each of 500 jobs regenerates its server's 400 shards one at a time into scratch space with the build's own code and the bootstrap's basis, assigns the rows to cells with the global coarse quantizer and adopts the bootstrap's radii, runs the exact scan and the routed search at 32 and 128 probes on that shard, keeps the best ten per query, and discards the shard. Merging top-ten lists across shards is exact, so the result equals a search of the whole server. Every regenerated shard file and its two cell files are hashed and compared with the sha256 the first run recorded for the released index. PENDING-1TRG-RESULTS.

## 4.8 Work per query

With 2048 fixed cells, a fixed probe width scores a roughly fixed share of the index, so recall that holds at a fixed width is bought with work that grows with the corpus. Table 7 gives that work. The routed pass scores every row of the probed cells on every server, so the rows scored for a query at width $p$ are the sizes of the first $p$ cells of its probe order, summed over the fleet, which the cell census gives exactly and which we checked against the probe orders the first run recorded. At 32 probes a corpus-row query scores a median of 19.2 billion rows, 1.92 percent of the index, against the 15.6 billion balanced cells would give, because the cells a query reaches first are larger than the median cell. At 128 probes it scores 72 billion. Every query touches all 500 servers, because at every width measured each server holds rows in some of the probed cells. At 24 bytes per row the median query at 32 probes touches 0.46 TB of codes, norms and cell membership. That is the payload the scan reads, not a device measurement. The non-member queries cost slightly less at every width, and so their lower recall at 32 probes is not a matter of less work. The routed jobs' times are no measure of this work's latency, because each job opens 400 shards from network block storage and serves 100 queries at once.

**Table 8: What one job of each kind used, one sampled job each, in the one-CPU, two-gigabyte class. Memory is what the job's control group is charged, which includes page cache. Own memory is the process's resident set, or its anonymous memory for the build.**

| job | wall | mean cores | mean mem. | peak mem. | own mem. |
|---|---|---|---|---|---|
| build | 2.5–3.8 h | | | 2048 MiB | 292 MiB |
| exact scan | 1912 s | 0.88 | 1242 MiB | 1469 MiB | 346 MB |
| routed pass | 36 min | 0.48 | 1258 MiB | 1568 MiB | 165 MB |

## 5 HOW THE MEASUREMENT RAN

*Jobs.* Every job in the build and the measurement requested one CPU and two gigabytes of memory, the cluster's smallest class, except 164 exact scans submitted in an earlier session at six CPUs and eight gigabytes. The pool resumed at one CPU, kept those 164 partials and scanned the other 336 servers. The median scan call took 2573 seconds at six CPUs and 2757 seconds at one, so six cores bought about seven percent on a scan that holds one. A scan over memory-mapped shards fits two gigabytes once three choices are made. The queries come from the cache, because deriving them from the seed regenerates four whole five-million-row blocks. At most two shards are open, because a mapped shard still materializes its row-id and tombstone arrays, 45 MB per shard, and the default of 128 open shards is 5.8 GB. Scoring proceeds in blocks of 65536 rows. A build job streams its shards from the seed in row bands and drops each written band from the page cache, because written pages count against the job's memory limit. Table 8 gives what one job of each kind used.

*Submission.* The driver publishes each job to a controller [3] that creates it only while the namespace has fewer than five jobs pending and twenty running and the cluster has fewer than a hundred pending pods, and retries a deferred job after waits that double from thirty seconds to fifteen minutes, for up to fifteen attempts, following an earlier design for polite submission to shared clusters [4]. Above the controller a pool keeps at most twenty jobs in flight with no wave barrier, checkpoints each phase to a state file, adopts jobs still on the cluster after a restart, recycles a failed job after a back-off from two minutes to thirty, and waits ninety minutes for a submitted job to appear before it treats the job as lost, because a job the controller has deferred is not yet visible and resubmitting it would queue a duplicate.

*Safety of repeated jobs.* Every job checks for its output and exits if it is there, writes its output under a temporary name and renames it into place. A duplicate, a retry and a job adopted after a restart therefore compute the same partial and either find it done or replace it whole. Two cases are weaker than that, and we state them. The temporary name was per output and not per attempt, and Kubernetes allows only one job of a name, but a pod can outlive its job on a node that no longer responds, so two live writers for one server were not excluded by construction. And the score phase loads every partial it finds, and the figure script asserts 500 per phase, but nothing checks shapes or duplicate files. The regenerated-index run of Section 4.7 writes under names unique to each attempt and loads every existing output before it treats the output as done.

**Table 9: The pool over the first run, 2026-09-25 18:01 UTC to 2026-09-27 08:59 UTC, and the second sweep, 2026-09-27 21:37 UTC to 2026-09-29 12:56 UTC. An earlier session at six CPUs per scan, whose 164 finished scans the first run kept, made 432 submissions more and is not counted here.**

| | first run | second sweep |
|---|---|---|
| submissions | 928 | 574 |
| completions | 849 | 501 |
| recycles after a failed job | 75 | 73 |
| storage read fault on one node | 24 | 25 |
| node lost during the job | 13 | 17 |
| node unreachable, pod stuck | 13 | 11 |
| container start failure | 2 | 1 |
| other | 23 | 19 |
| recycles after 45 minutes pending | 26 | 0 |
| submissions deferred by the controller | 1 | 0 |

After the runs we audited the archived partials of both query sets with a script in the repository (`fleet_partials_audit.py`). It checks that each phase has exactly one partial for each of the 500 servers and no stray or temporary file beside them, that every partial holds ids and scores of shape 100 by 10 in query-file order, that every id lies in its server's row range and none repeats within a query, that scores are finite and non-increasing, that a row in both a server's reference and routed partials has the same score up to floating-point reordering, that no routed score exceeds the reference score of the same rank by more than that reordering difference, and that the merged reference top-10 of every query has no tie at its boundary and a gap there wider than twice the largest score difference. Both runs pass, 3000 partials of the corpus-row run and 1500 of the non-member run, the latter also against the sha256 of every archived file. Partials record no query identifier, so query order is checked by shape against the shared query file and not row by row.

*Failures.* Table 9 gives what the pool did. Two failure classes dominated. One node returned a storage read fault to every job placed on it within 3 to 13 minutes of start, and retries landed elsewhere. Pods pending past 45 minutes were recycled in the first run, and in every instance checked the original pod had run and the retry found its output within minutes. In the second sweep that rule was replaced by waiting, because a running job is never deleted on a namespace other groups share. Six pods left on nodes that no longer responded were removed by hand across the four days. Six index volumes became unreachable when their node went offline, and each was recreated and rebuilt from its seed in 2 hours 32 minutes to 3 hours 49 minutes. No backup of the index was kept.

*Provenance.* The phase scripts came from a configuration map fixed for each run, whose scan and routed scripts match the repository at commit 8f25bc4 byte for byte. The library and the merge came from a clone of the default branch at each job's start, and twelve commits, c434a99 to 0b5bd07, appear in the pod logs of the first run. Between those two commits the corpus generator, the shard format, the routed search, the probe order and the score program are unchanged. The one library file on the path that changed,

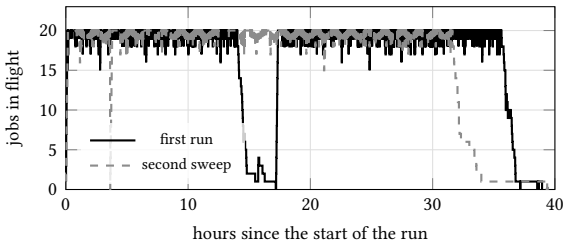



**Figure 5: Jobs the pool had in flight, from the driver logs, for the first run (exact scan, routed passes at 32 and 128 probes, score) and the second sweep (routed passes at 16, 64 and 256 probes, score). The median is nineteen in both.**

**Table 10: Search time per server at $10^{12}$ rows, corpus-row queries, 500 servers per phase, and its sum over servers. Each time is the job's search call, which opens 400 shards and reads them from network block storage. It excludes the pod's start, image pull, dependency install and clone. The three routed widths of the second sweep ran inside one job per server.**

| phase | median | mean | p10 | p90 | max | sum (h) |
|---|---|---|---|---|---|---|
| exact scan | 2664 s | 3008 s | 1850 s | 4009 s | 16308 s | 418 |
| routed, 32 | 1064 s | 1131 s | 775 s | 1466 s | 14286 s | 157 |
| routed, 128 | 1335 s | 1420 s | 958 s | 1861 s | 14524 s | 197 |
| routed, 16 | 916 s | 1071 s | 660 s | 1440 s | 12236 s | 149 |
| routed, 64 | 1219 s | 1366 s | 912 s | 1743 s | 14645 s | 190 |
| routed, 256 | 1594 s | 1719 s | 1176 s | 2199 s | 13786 s | 239 |

the asymmetric-distance index, gained a tracing hook that is off unless enabled, a property and a refactor of its rerank, none on the memory-mapped scan the fleet uses. A per-job record of the commit exists only for recycled pods, the pilots and the score job, because the driver keeps output only for those. The regenerated-index run uses one staged copy of the build's dependencies for every job and logs its hash.

## 6 WHAT THE RUN USED

Table 10 gives the search time of every successful measurement job. Summed over servers it is 772 hours for the first run, the exact scan and the routed passes at 32 and 128 probes, and 577 for the second sweep, 1349 hours for the corpus-row queries. The non-member run adds 835, of which its exact scan is 437. Those sums are not CPU consumption, and we keep four quantities apart. The first is that search time, 1349 hours. The second is what the jobs held during it. The 164 exact scans at six CPUs account for 145.6 of the scan's 417.8 hours, so the scan held 1146 core-hours over its search calls and the whole corpus-row measurement 2077, and those figures leave out each pod's start and the time between its search calls, which the record does not hold. The third is what the jobs used. One sampled job of each kind averaged 0.88 cores for the scan and 0.48 for a routed pass (Table 8), and no fleet-wide metering was recorded. The fourth is work that produced nothing kept. The first run made 928 submissions for 849 completions and the second sweep 574 for 501, and the earlier session at six CPUs made 432 submissions with 260 recycles. The record gives the count of those attempts and not their duration. The build is accounted separately. The six servers rebuilt in September took 2 hours 32 minutes to 3 hours 49 minutes each at one CPU, and the build pool of August and September kept no per-server log, so its total is not in the record. Storage is 24.0 TB of index on 30.1 TB of provisioned block volumes.

The routed times are not the latency of inverted-file search, and their ratio to the scan time is not a speedup of the method. Each routed job opens its 400 shards from network block storage and reads the probed cells for 100 queries at once, and Section 4.8 gives the work. The tails are node variance. Between 4 and 30 servers per phase took more than twice the median, the 16308 s maximum of the scan is one server on one node that also produced the slowest routed pass, and in every phase the ninetieth percentile is within 1.6 times the median.

## 7 WHAT THE RESULT DOES AND DOES NOT SHOW

Recall of the routed shortlist against the exact scan of the same compressed index does not degrade from $10^8$ to $10^{12}$ rows for corpus-row queries, and at $10^{12}$ rows queries outside the index give 0.969 at 32 probes and 0.999 at 128. That is a statement about the inverted-file layer, its shared coarse quantizer and its merge. Table 11 lists what could make the result wrong or narrower than it reads, and what in the record addresses each.

It is not a statement about recall against the truth. The compressed scan at $10^9$ rows found 0.592 of the float neighbours, and on a real corpus of 15 million Cohere Embed v3 vectors the same index found 0.693 of them by exact scan, 0.689 by routing at 256 probes, and 0.981 after reranking the routed shortlist against stored originals. On that corpus the routed shortlist returned 0.990 of the compressed scan at 256 probes, so the smaller real-data pilot supports the routing result, and the compression result does not carry over. Four bits per dimension at this rate loses neighbours that only a rerank tier recovers, and the trillion-row corpus, at intrinsic dimension 16 against a real embedding's 52 to 61, cannot show that.

A cold store of originals at $10^{12}$ rows would be 128 TB of float vectors, so true recall is established at $10^9$ and at the real pilot, not here. The seeded corpus does allow one bound. The float rows named by a shortlist can be regenerated from the seed, so the compressed top-10 of each query can be reranked by float cosine within the shortlist that contains it, and the share of the compressed top-10 that survives that rerank is an upper bound on compressed-only true recall, because a corpus float neighbour that lies in the shortlist is a shortlist float neighbour. The set inequality behind the bound is checked in Lean 4 with Mathlib (`lean/Pvldb1t/RerankBound.lean`, theorem `rerank_upper_bound`), under the same caveat as the identity, that the proof covers the set relationship and not the data. Table 12 gives the result on 11002 regenerated rows. Of the reference scan's compressed top-10, 0.621 survives the float rerank of its own shortlist on average, with 22 of 100 queries keeping all ten and a minimum of 0.100, so compressed-only true recall at $10^{12}$ rows is at most 0.621 on this corpus. At one billion rows the cold store gave 0.592, inside this bound. The routed shortlists behave the same way, 0.623 at 16 probes and 0.621 at 256. After reranking, the routed top-10 agrees with the reranked reference top-10 at 0.940 for 16 probes, 0.982 for

**Table 11: Threats to validity and what addresses each.**

| threat | what addresses it |
|---|---|
| The corpus is synthetic, intrinsic dimension about 16 against 52 to 61 for real embeddings | On 15 million Cohere Embed v3 vectors the routed shortlist returned 0.990 of the compressed scan at 256 probes, which supports the routing result on real data at that size. The compression result does not carry over, and the next row bounds it. |
| The reference is the compressed index's own exact scan, not float ground truth | At $10^9$ rows the compressed scan found 0.592 of the float neighbours. At $10^{12}$ the rerank bound puts the compressed ranking at no more than 0.621 of the true neighbours. |
| Each corpus-row query is its own nearest neighbour | On 100 queries outside the index recall was 0.969 at 32 probes and 0.999 at 128 (Section 4.6), and their nested series rises with the rows as the corpus rows' does (Section 4.2). PENDING-1TRG-THREAT-SELF |
| The result could depend on the router's radius adjustment | At equal rows scored, plain centroid order matches or beats it (Section 4.5). |
| Partials could be duplicated, malformed or inconsistent | An audit of all 4500 archived partials checks count, shape, id range, ordering, cross-pass scores and merge ties, and both runs pass (Section 5). |
| Queries come from four shards, so four of the generator's subspaces | PENDING-1TRG-THREAT-SHARDS |
| One hundred queries per set | Bootstrap intervals over queries of 0.982 to 0.995 at 32 probes and 0.997 to 1.000 at 128 for the corpus rows, and 0.954 to 0.982 and 0.997 to 1.000 outside the index. They describe the sample, not other subspaces. |
| Scale is confounded with the index across runs | The nested series varies only the number of rows inside one index (Section 4.2). |
| High recall at a fixed width could hide growing work | Rows scored per query are given at every width (Table 7). |
| Code changed during the run | The phase scripts came from a fixed configuration map, and across the twelve commits the jobs cloned nothing on the path of generation, scanning, routing or scoring changed (Section 5). |

**Table 12: Rerank bound on regenerated float rows, 100 corpus-row queries, shortlists of width 100 cut from the merge of per-server top-10 partials. Survival is the share of the compressed top-10 in the float top-10 of the same shortlist, an upper bound on compressed-only true recall. Agreement is the share of the routed shortlist's float top-10 that is in the reference shortlist's float top-10, an agreement between restricted candidate sets.**

| shortlist | ref. | 16 | 32 | 64 | 128 | 256 |
|---|---|---|---|---|---|---|
| survival of the top-10 | 0.621 | 0.623 | 0.623 | 0.624 | 0.622 | 0.621 |
| agreement after rerank | – | 0.940 | 0.982 | 0.996 | 1.000 | 1.000 |

32, 0.996 for 64 and 1.000 from 128 on, within 0.012 of the recall before reranking at every width. That agreement is between two reranked shortlists of width 100, each a restricted candidate set, and it is not float recall over the corpus.

## 8 RELATED WORK

Product quantization with asymmetric distance computation and inverted files [12] is the basis of the index, and the two-stage arrangement in which a compressed scan produces a shortlist that a finer code or the original vectors rerank dates from the same group's billion-vector work [13]. Locally optimized product quantization [15] learns a separate rotation and codebook for each cell, the opposite choice to the shared basis we use to keep scores comparable across shards, and anisotropic quantization [10] and RaBitQ [8] are the current forms of the quantizer with, in the latter case, an error bound on the estimated distance. Locally adaptive scalar quantization [2] puts a compressed representation under a graph index and reranks from a second tier, the arrangement our $10^9$ and real-embedding figures use. Locality-sensitive hashing [11] and navigable graphs [16] are the usual alternatives to the inverted file and are not what this paper measures.

Billion-scale search on one machine or one node's disk is established [5, 14, 20], and the NeurIPS'21 Big ANN competition [19] fixed the datasets and the reporting for that scale, with its 2023 successor [18] turning to filtered, sparse, out-of-distribution and streaming search at ten million rows. Distributed vector databases such as Milvus [22] and its cloud-native successor [9] shard an index across nodes and merge per-node results, the arrangement our fleet uses without a serving layer, recent work places a billion-scale index on distributed storage [24], and a single graph index has been spread over more than a thousand machines at $5 \times 10^{10}$ vectors [1].

The closest precedents are at our scale. The Faiss library paper [6] indexes 1.5 trillion 144-dimensional vectors, compressed to 54 bytes each, built as 2000 shards on a few hundred servers and regrouped into 100 indexes by inverted list, memory-mapped on one machine and searched through 20 intermediate servers in about a second per query. It reports the construction and the search time and no recall at that scale. Its Section 5.1 gives the argument for why recall at a fixed probe count should not fall as a fixed-quantizer index grows, with evidence on up to a billion vectors in the regime of more cells than the data needs, and our nested series is a measurement of that argument at a trillion rows in the opposite regime. UBASE [21] serves about a trillion vectors at ByteDance and reports latency and memory for that deployment and recall on sets of up to about a hundred million vectors. Both are systems that run at a trillion vectors. This paper is a measurement at a trillion rows. We found no published comparison of a routed index against the exact scan

of the same index at that scale, no scale series inside one index, and no record from which such numbers can be recomputed.

## 9 REPRODUCIBILITY

The repository snapshot at the availability URL, archived as DOI 10.5281/zenodo.23608470, carries the corpus seeds and generator, the build, scan, routed, score and metadata scripts, the pool driver, and under `benchmarks/fleet/record/1t/` the pool state files, every driver log, the pilot logs, the interim score and the score logs whose RESULT_JSON lines hold every per-server search time. The figures and tables are produced from those logs by `fig_data.py` in `paper/pvldb1t/figdata`, the per-query figures for the non-member set by `fleet_nonmember_analysis.py`, the work per query by `fleet_work_analysis.py`, the nested series by `fleet_nested.py`, the cell-order control by `fleet_order_control.py` and the audit by `fleet_partials_audit.py`, all in `benchmarks/fleet`. The two scale-transfer registrations, their predictions, the hashes of their grader scripts and the grades are under `docs/` and `record/1t/post/`, and the git history shows each prediction committed before the score. The shared volume of the run, with the bootstrap shard, the coarse quantizer, every per-server partial and the cell census, and the partials and per-volume index checksums of the non-member run, are archived as DOI 10.5281/zenodo.23608434, each file with its sha256. PENDING-REPRO-1TRG. The Lean development under `paper/pvldb1t/lean/` builds against Mathlib at Lean 4.32.2, and its axiom audit, recorded in `AXIOMS.txt`, shows every theorem depending on propositional extensionality, quotient soundness and choice only.

## 10 CONCLUSION

A trillion-row compressed index was scanned exactly and searched by routing over the same shards. Scaling from $10^8$ to $10^{12}$ rows causes no measurable loss of routed recall against the exact scan for corpus-row queries, inside the one index recall at narrow widths rises with the number of rows, and queries the index does not contain give 0.969 at 32 probes and 0.999 at 128. PENDING-1TRG-CONCLUSION. Recall at any width is the share of reference neighbours whose cell the router reaches, a count that matched five measured widths, and by that count plain centroid order does as well as the router's own at equal work. The work that buys the recall is a fixed share of the index per query, a median of 19 billion rows at 32 probes, so a width tuned small held as the corpus grew, and bounding the work would take more cells. A registered fit on twenty servers predicted the non-member recall at $10^{12}$ rows within error, and a registered fit on separate small corpora failed at 32 probes. Against float rows regenerated from the seed, the compressed ranking keeps at most 0.621 of the true neighbours. All of it ran in a shared cluster's smallest job class and can be recomputed from the archived record.

## ACKNOWLEDGMENTS

This work used the National Research Platform's Nautilus cluster. The National Research Platform is supported in part by National Science Foundation awards CNS-1730158, ACI-1540112, ACI-1541349, OAC-1826967, OAC-2112167, CNS-2100237 and CNS-2120019. We thank its operators and community.

## REFERENCES

[1] Philip Adams, Menghao Li, Shi Zhang, Li Tan, Qi Chen, Mingqin Li, Zengzhong Li, Knut Risvik, and Harsha Vardhan Simhadri. 2025. DistributedANN: Efficient Scaling of a Single DiskANN Graph Across Thousands of Computers. *arXiv preprint arXiv:2509.06046* (2025). https://arxiv.org/abs/2509.06046

[2] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore Willke. 2023. Similarity Search in the Blink of an Eye with Compressed Indices. *Proceedings of the VLDB Endowment* 16, 11 (2023), 3433–3446. https://doi.org/10.14778/3611479.3611537

[3] Andrew H. Bond. 2026. nats-bursting: submitting batch jobs to a shared Kubernetes cluster over a NATS bus, with politeness back-off. https://github.com/ahb-sjsu/nats-bursting.

[4] Andrew H. Bond. 2026. polite-submit: client-side contention management for Slurm clusters with carrier-sense back-off. https://github.com/ahb-sjsu/polite-submit.

[5] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. Curran Associates, Inc., Red Hook, NY, USA.

[6] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. https://doi.org/10.1109/TBDATA.2025.3618474

[7] Elena Facco, Maria d'Errico, Alex Rodriguez, and Alessandro Laio. 2017. Estimating the Intrinsic Dimension of Datasets by a Minimal Neighborhood Information. *Scientific Reports* 7 (2017), 12140. https://doi.org/10.1038/s41598-017-11873-y

[8] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 3 (2024). https://doi.org/10.1145/3654970

[9] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3548–3561. https://doi.org/10.14778/3554821.3554843

[10] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research)*, Vol. 119. PMLR, 3887–3896. https://proceedings.mlr.press/v119/guo20h.html

[11] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, New York, NY, USA, 604–613. https://doi.org/10.1145/276698.276876

[12] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. https://doi.org/10.1109/TPAMI.2010.57

[13] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in One Billion Vectors: Re-rank with Source Coding. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 861–864. https://doi.org/10.1109/ICASSP.2011.5946540

[14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. https://doi.org/10.1109/TBDATA.2019.2921572

[15] Yannis Kalantidis and Yannis Avrithis. 2014. Locally Optimized Product Quantization for Approximate Nearest Neighbor Search. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2329–2336. https://doi.org/10.1109/CVPR.2014.298

[16] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

[17] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. 2010. Hubs in Space: Popular Nearest Neighbors in High-Dimensional Data. *Journal of Machine Learning Research* 11, 86 (2010), 2487–2531. https://jmlr.org/papers/v11/radovanovic10a.html

[18] Harsha Vardhan Simhadri, Martin Aumüller, Matthijs Douze, Dmitry Baranchuk, Amir Ingber, Edo Liberty, George Williams, Ben Landrum, Magdalen Manohar, Mazin Karjikar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Weiguo Zheng, Yizhuo Chen, Jie Yin, and Ben Huang. 2025. Results of the Big ANN: NeurIPS'23 Competition. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, Vol. 38.

Curran Associates, Inc., Red Hook, NY, USA, 76394–76408. https://doi.org/10.52202/085713-2303

[19] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (Proceedings of Machine Learning Research)*, Vol. 176. PMLR, 177–189. https://proceedings.mlr.press/v176/simhadri22a.html

[20] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA.

[21] Yao Tian, Yuncheng Lu, Liyao Xiong, Yuming Xu, Hao Zhang, Weichen Zhao, Xi Zhao, Bo Kuang, Dongyu Wang, Jiehui Li, Yakun Li, and Lei Zhang. 2026. UBASE: An AI Search Engine for Trillion-Scale Vector Data Management at ByteDance. *arXiv preprint arXiv:2608.30607* (2026). https://arxiv.org/abs/2608.30607

[22] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, USA, 2614–2627. https://doi.org/10.1145/3448016.3457550

[23] Derek Weitzel, Ashton Graves, Sam Albin, Huijun Zhu, Frank Würthwein, Mahidhar Tatineni, Dmitry Mishin, Elham E. Khoda, Mohammad Firas Sada, Larry Smarr, Thomas DeFanti, and John Graham. 2025. The National Research Platform: Stretched, Multi-Tenant, Scientific Kubernetes Cluster. In *Practice and Experience in Advanced Research Computing 2025: The Power of Collaboration (PEARC '25)*. ACM, New York, NY, USA. https://doi.org/10.1145/3708035.3736060

[24] Kun Yu, Jiabao Jin, Xiaoyao Zhong, Peng Cheng, Lei Chen, Zhitao Shen, Jingkuan Song, Hengtao Shen, and Xuemin Lin. 2025. Approximate Nearest Neighbor Search of Large Scale Vectors on Distributed Storage. *arXiv preprint arXiv:2510.17326* (2025). https://arxiv.org/abs/2510.17326